Windows 8, 10, 2012 Server TCP/IP Tweaks

Tweak TCP/IP in Windows 8,10,2012 to speed up your broadband internet
2014-12-12 (updated: 2016-03-15) by Philip
Tags: CTCP, Windows 8, TCP/IP, PowerShell, tweaks, Chimney Offload, TCP Window, ICW, netsh, DCA, Windows 10, RWIN



The Windows 8 TCP/IP implementation shares many traits with earlier Windows versions, however, there are some subtle differences, new syntax for applying tweaks, and some new settings. Windows 8 generally works well by default for many broadband connections, yet there is always some room for improvement and we will try to cover all relevant settings and their effect on your connection in detail below.

To run some of the outlined commands, you will need to use an administrator account. With earlier Windows versions, we used the netsh command in *elevated command prompt* to tweak most of the TCP/IP settings. With the introduction of Windows 8/Server 2012, some of those parameters are only tweakable by using the *Windows PowerShell*, that is why we will be using it for settings when possible. Note that netsh and other DOS commands are still available under *PowerShell* , just as with *Command Prompt*.

The settings outlined bellow work with Windows 8, 8.1, Windows 10 Technical Preview, Windows Server 2012, and 2012 R2 (with only minor syntax differences). If you'd rather use our free software to apply all those settings, try the SG TCP Optimizer (v.4).

Windows PowerShell

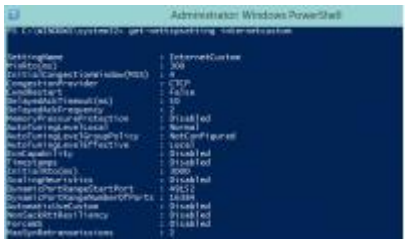
The *Windows PowerShell* is a tool developed by Microsoft designed to replace and extend the functionality of the *command prompt*. It allows you to run all DOS commands similarly to *command prompt*, however, it also provides additional functionality in the form of many customizable commands called *cmdlets*. We will be using *PowerShell* to tweak TCP/IP settings, as some of the parameters are no longer tweakable using *command prompt netsh* in Windows 8 and newer OSes.

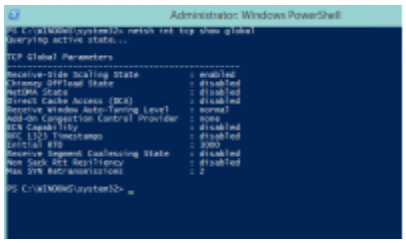
In Windows 8, the easiest way to find *PowerShell* is by searching for PowerShell on your *Start* screen/menu. It can also be accessed from the *Windows System* folder. You should right-click on *PowerShell* and choose "Run as administrator" so that you have sufficient permissions to tweak all applicable settings.

The PowerShell commands below are an overview of what we can use to tweak TCP/IP settings under Windows 8:

Get-NetTCPSetting - this can be used to view TCP parameters
Set-NetTCPSetting - this can be used to set TCP parameters

netsh int tcp show global - you can still see and change some (not all) parameters using netsh





There are a few different networking profiles called "templates" in Windows 8 PowerShell under Get-NetTCPSetting.
In Windows 8 (and 2012 Server): Automatic, Internet, Datacenter, Compat
In Windows 8.1 (also Windows 10, Windows 2012 Server R2): Automatic, Internet, InternetCustom, Datacenter, DataCenterCustom, Compat

Theoretically, only one of the above templates can be modified by default - "Custom" under Windows 8, "InternetCustom" under Windows 8.1 (and Windows 10). The "DatacenterCustom" template can only be modified if Windows is installed on a server. If you try to modify one of the other templates, you will get an error, something like:

Set-NetTCPSetting : Only the custom templates InternetCustom and DatacenterCustom can be modified.

Even though theoretically only the "Custom" templates can be modified, many of the commands below (both netsh and PowerShell TCP cmdlets) are global and modify all templates simultaneously.

By default, the "Internet" template/profile is applied to TCP connections. To find the currently used template using PowerShell cmdlets type either:

Get-NetTCPConnection , or **Get-NetTransportFilter**
(using netsh, you can type: netsh int tcp show supplemental)

Notes:
PowerShell cmdlets are case-insensitive, so typing get-nettcpsetting is the same as Get-NetTCPSetting.

Even though some settings below modify all "templates", it may still be necessary to sometimes change the default template used for TCP connections from Internet to Custom/InternetCustom. The following command supposedly changes the default (does not

work as documented by MS under Windows 8/8.1):
set-NetTCPConnection -AutomaticUseCustom true
(acceptable parameters are [true|false], doesn't seem to work in Windows 8.1 ?)

Useful TCP/IP connection related cmdlets:
Get-Help Get-NetTCPSetting -detailed (help on NetTCPSetting)
Get-NetTCPConnection (see active connections)
(Get-NetTransportFilter | Where DestinationPrefix -eq '*' | Get-NetTCPSetting) -- view current template settings
(Get-NetTransportFilter | Where DestinationPrefix -eq '*' | Get-NetTCPSetting).CongestionProvider -- view only "CongestionProvider" setting in currently used template

See also: MS TechNet Set-NetTCPSetting

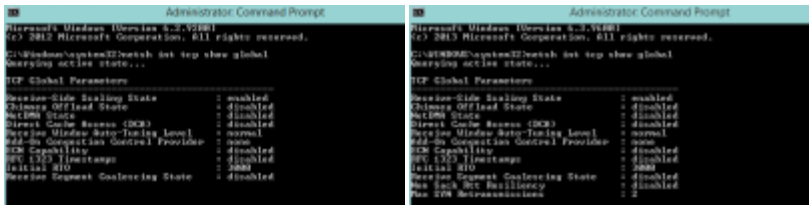
Netsh

It is still possible to check the current status of Windows TCP/IP parameters with netsh, both in *PowerShell* and *elevated command prompt*. We recommend using the PowerShell for all changes. While most settings can still be changed using the "netsh" tool, others require the Windows PowerShell interface and cmdlets. Netsh (and the command prompt) is being deprecated by Microsoft, and some of the netsh commands are bugged in Windows 8/8.1 and Server 2012/2012 R2.

To see current tcp parameters in netsh, use:

netsh int tcp show global

You will see something like the following:



There are two extra settings in Windows 8.1 and Windows 10 (right image above)

Explanation of all the tweakable TCP/IP parameters, possible ways to change them, as well as their default/recommended states for broadband internet connections are listed below. We recommend using the Windows *PowerShell* for all changes. The parameters are loosely sorted in order of importance with the parameters that generally have more significant impact listed first.

Receive Window Auto-Tuning Level

The TCP Receive Window has a very important role in TCP connections. It can limit throughput, especially in high-speed, high-latency environments, such as most internet connections. For more information on how it limits your speed, check our TCP Window article.

The possible settings are as follows:

disabled: uses a fixed value for the tcp receive window. Limits it to 64KB (limited at 65535).
highlyrestricted: allows the receive window to grow beyond its default value, very conservatively
restricted: somewhat restricted growth of the tcp receive window beyond its default value
normal: default value, allows the receive window to grow to accommodate most conditions
experimental: allows the receive window to grow to accommodate extreme scenarios (not recommended, it can degrade performance in common scenarios, only intended for research purposes. It enables RWIN values of over 16 MB)

Default setting: normal
Our recommendation: normal (unless you're experiencing problems)

To modify, in *PowerShell* (or elevated command prompt) type:

netsh int tcp set global autotuninglevel=normal
(where "normal" is one of the above listed possible states)

Same can also be accomplished using *PowerShell* cmdlets, just type:

Set-NetTCPSetting -SettingName InternetCustom -AutoTuningLevelLocal Normal

Note: You can also try "highlyrestricted" autotuninglevel for up to 10Mbps connections, as it actually uses a higher unscaled RWIN value (16384 vs. 256 bytes), and still scales up to 262140 bytes.

Disable Windows Scaling heuristics

Windows 8 (as with Windows 7) has the ability to automatically change its own TCP Window auto-tuning behavior to a more conservative state regardless of any user settings. When heuristics restricts your autotuning level, you may see this message when viewing netsh settings:

"** The above autotuninglevel setting is the result of Windows Scaling heuristics overriding any local/policy configuration on at least one profile."

When heuristics restricts autotuning level, the "netsh int tcp show global" command will still (incorrectly) show your user-set autotuning level, you have to use "netsh int tcp show heuristics" to see the actual current heuristics restriction. To enforce the user-set TCP Receive Window autotuning level and prevent heuristics from restricting how the TCP Receive Window grows, disable heuristics. It is best to disable this before applying autotuning level to ensure your user-set autotuning level is retained.

default state: OS-dependent (disabled in Windows 8.1, enabled in Windows 7)
possible settings are: disabled,enabled,default (sets to the Windows default state)
recommended: disabled (to retain user-set RWIN auto-tuning level)

To disable explicitly using netsh:

netsh int tcp set heuristics disabled

To see the current heuristics state:

netsh int tcp show heuristics

Using PowerShell cmdlets:

Set-NetTCPSetting -SettingName InternetCustom -ScalingHeuristics Disabled

Notes:
If heuristics are not explicitly disabled, Windows can restrict the RWIN autotuninglevel at any time. When that happens, viewing your settings with *"netsh int tcp show global"* will still incorrectly show user-set autotuninglevel, only *"netsh int tcp show heuristics"* reveals the restriction.

The registry value that was used in Windows 7 and earlier is no longer present in the Windows Registry
(~~HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\services\Tcpip\Parameters\EnableWsd=0~~ (not present for Windows 8))

Add-On Congestion Control Provider (CTCP)

The traditional slow-start and congestion avoidance algorithms in TCP help avoid network congestion by gradually increasing the TCP window at the beginning of transfers until the TCP Receive Window boundary is reached, or packet loss occurs. For broadband internet connections that combine high TCP Window with higher latency (high BDP), these

algorithms do not increase the TCP windows fast enough to fully utilize the bandwidth of the connection.

Compound TCP (CTCP) is a newer congestion control method that increases the TCP Send Window more aggressively for broadband connections (with large RWIN and BDP). CTCP attempts to maximize throughput by monitoring delay variations and packet loss. It also ensures that its behavior does not impact other TCP connections negatively. CTCP is turned on in Windows 8, and Server editions by default, but off in Windows 7. Keeping CTCP on can significantly increase throughput and packet loss recovery.

Possible settings are: none, ctcp, dctcp (Datacenter TCP)
Default : CTCP ** (netsh incorrectly shows "none" by default, while PowerShell Get-NetTCPSetting shows "CTCP". netsh is deprecated and incorrect)
Recommended: CTCP

CTCP - Compound TCP increases the receive window and amount of data sent. It can improve throughput on higher latency/broadband internet connections.
DCTCP - Data Center TCP adjusts the TCP Window based on network congestion feedback based on *Explicit Congestion Notification (ECN)* signaling, it is designed to improve throughput on low latency/local links.

Unfortunately, netsh is bugged for this setting, and the PowerShell cmdlet only works for server Windows variants, so there is no longer an easy one line command that can change the congestion provider. Yay for Microsoft! Fortunately, the default setting is already CTCP, as per the *PowerShell* **Get-NetTCPSetting** cmdlet.

To change using *PowerShell* cmdlet:

Set-NetTCPSetting -SettingName InternetCustom -CongestionProvider CTCP

("InternetCustom" template is for Windows 8.1, change to "Custom" for Windows 8)

To see the current Congestion provider, use: *Get-NetTcpSetting*
Alternatively, you can go wild and use the cmdlet below to see only the congestion provider of the currently used template:
(Get-NetTransportFilter | Where DestinationPrefix -eq '' | Get-NetTCPSetting).CongestionProvider*

Notes:
Changing CTCP directly with netsh is not possible by default under Windows 8, those commands are listed just for reference:

netsh int tcp show supplemental (see the currently used supplemental template)
netsh int tcp set supplemental custom 300 10 ctcp disabled 50 (minrto=300ms, initial congestion window icw=10 packets, congestionprovider=ctcp, enablecwndrestart=disabled, delayedacktimeout=100 ms) netsh int tcp set supplemental custom (tell Windows to use the custom template)

You can still "hack" Windows 8 to show CTCP in netsh by doing the following:

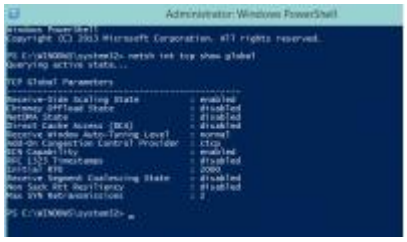
1. Open notepad and copy this code into it:

```
Windows Registry Editor Version 5.00
[HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Nsi\{eb004a03-9b1a-11d4-9123-0050047759bc}\0]
"0200"=hex:00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,\
00,00,00,00,00,00,02,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,\
00,00,00,ff,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,\
ff,00,00,00,00,00,00,00,00,00,00,00,00,00,00
"1700"=hex:00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,\
00,00,00,00,00,00,02,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,\
00,00,00,ff,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,\
ff,00,00,00,00,00,00,00,00,00,00,00,00,00,00
```

2. Save file with .reg extension, for example: "enable-ctcp.reg"
3. Import the registry file into the Windows Registry (double-clicking on it should do it, after a warning).
4. Reboot

Alternatively, get **[this file]** (right-click, "save target as", save with .reg extension so that it can be merged with the registry automatically when double-clicked).

Here is a screenshot from Windows 8.1 after merging the above with the registry and rebooting:



You can also just copy all the text to the appropriate registry hive directly. Merging the above with the registry and rebooting will show CTCP as the addon congestion control algorithm. Still, as per Microsoft, netsh is deprecated and you should be using Powershell.

Note that the above registry hack will change not only CTCP, but other related parameters as well, here is some additional info:

The binary values above affect many parameters: "Chimney Offload State" (offset 04, 00=disabled,01=enabled,02=auto), RSS (offset 05, 00=default/enabled,01=disabled), ECN Capability (offset 06, 00=disabled/default, 01=enabled), CTCP (offset 28, 00=none, 02 = ctcp), etc.

TCP Chimney Offload

TCP chimney offload enables Windows to offload all TCP processing for a connection to a network adapter (with proper driver support). Offloads are initiated on a per-connection basis and reduce networking-related CPU overhead, theoretically enabling better overall system performance by freeing up CPU time for other tasks. TCP Chimney Offload controls what is referred to as the "TCP Offload Engine" (TOE) in Windows in general. It is a global setting that has to be enabled for many of the other offloads to work. Enabling this setting had some negative effects in the past because of buggy network adapter drivers, however its implementation has gotten much better with time. It is useful for CPU-bound client computers and very fast broadband connections, not recommended in some server environments.

All the possible states of this setting are as follows:

automatic - offloads if the connection is 10 GbE, has a RTT default - this setting restores chimney offload to the system default.
disabled - chimney offload disabled.
enabled - chimney offload enabled.

Default: automatic
Recommended: disabled

To disable TCP Chimney Offload:

netsh int tcp set global chimney=disabled

Using Powershell, to set the offload state:

Set-NetOffloadGlobalSetting -Chimney Disabled (default: disabled, possible states: enabled/disabled/automatic, recommended: disabled*)

To view the current chimney state:

(Get-NetOffloadGlobalSetting).Chimney

or, simply:

Get-NetOffloadGlobalSetting

Notes:
* There were issues with buggy Network adapter drivers and the implementation of offloading with older OSes (Windows 2003 Server), however, this has improved over time. One should be more careful using offloading in server environments, as there have been some reports of issues with TCP Chimney Offload and SQL servers under heavy load, affecting both application concurrency and throughput. See this MSDN reference and MS KB942861.

Chimney Offload does not work together with NetDMA (mute point with Windows 8/8.1 as NetDMA is not supported).

Chimney Offload may not be supported with NAT/ICS (reference: Technet TCP Chimney Offload Compatibility for Windows 2008 R2). However, this only applies if NAT/ICS is enabled on the computer running Windows itself, and that is rarely the case.

Setting Chimney Offload to disabled is recommended for VMWare servers, and the setting is now considered deprecated by Microsoft. See: Obsolete RFCs and Overview of TCP Timers

~~NetDMA State~~

NetDMA (TCPA) enables support for advanced direct memory access. In essence, it provides the ability to more efficiently move network data by minimizing CPU usage. NetDMA frees the CPU from handling memory data transfers between network card data buffers and application buffers by using a DMA engine.

Default: disabled
Recommended: leave alone, don't bother (setting not supported in Windows 8 and later, according to MS)

You can still edit it using:

```
netsh int tcp set global netdma=disabled
```

Notes:
According to Microsoft, NetDMA is not supported in Windows 8/8.1 and changing this setting will have no effect. See: MSDN NetDMA

NetDMA (TCPA) **does not work together with Chimney Offload** either, you have to choose one or the other. For NetDMA to work, it must be enabled/supported by your BIOS and your CPU must support Intel I/O Acceleration Technology (I/OAT).

Direct Cache Access (DCA)

Direct Cache Access (DCA) allows a capable I/O device, such as a network controller, to deliver data directly into a CPU cache. The objective of DCA is to reduce memory latency and the memory bandwidth requirement in high bandwidth (Gigabit) environments. DCA requires support from the I/O device, system chipset, and CPU(s).

Possible states are: enabled, disabled
Default state: disabled
Recommended: enabled (provided the CPU/Chipset/NIC support it)

To set DCA using netsh:

```
netsh int tcp set global dca=enabled
```

Note: The impact of DCA is more significant with older CPUs.

Checksum Offload

The PowerShell Enable-NEtAdapterChecksumOffload cmdlet enables checksum offloads on the network adapter. Checksum offloading is also required for some other stateless offloads to work, including Receive Side Scaling (RSS), Receive Segment Coalescing (RSC), and Large Send Offload (LSO).

Default state: adapter-dependent
Recommended: enabled

In PowerShell:

Enable-NetAdapterChecksumOffload -Name * (use to enable checksum offload on all adapters that support it)
Disable-NetAdapterChecksumOffload -Name * (use to disable checksum offload for all network adapters)
Get-NetAdapterChecksumOffload (use to view network adapters that support checksum offload and their state)

The above PowerShell cmdlet example enables checksum offloads on all visible network adapters for all UDP/TCP/IP version 4 and version 6 traffic (and restarts the network adapters). Alternatively, you can specify only a single network adapter and/or protocol by using command-line switches. For more information on customizing the command, refer to this Technet article. To pick a single adapter and only modify its checksum offload state, find installed adapters using this cmdlet: *Get-NetAdapter*

Receive-Side Scaling State (RSS)

The receive-side scaling setting enables parallelized processing of received packets on multiple processors, while avoiding packet reordering. It avoids packet reordering by separating packets into "flows", and using a single processor for processing all the packets for a given flow. Packets are separated into flows by computing a hash value based on specific fields in each packet, and the resulting hash values are used to select a processor for processing the flow. This approach ensures that all packets belonging to a given TCP connection will be queued to the same processor, in the same order that they were received by the network adapter.

Default: enabled
Recommended: enabled (provided you have 2 or more processor cores and a NIC that can handle RSS)

To change RSS using netsh:

```
netsh int tcp set global rss=enabled
```

To change using PowerShell cmdlets:

Enable-NetAdapterRss -Name * (use this to enable RSS for all adapters that support it)
Disable-NetAdapterRss -Name * (use this to disables RSS for all adapters)
Get-NetAdapterRss -Name * (use cmdlet to view RSS capable Network Adapters)

Notes:
Needs *Checksum Offload* to be enabled. Only supported by some network adapters.

Receive Segment Coalescing State (RSC)

Receive Segment Coalescing (RSC) allows the NIC to coalesce multiple TCP/IP packets that arrive within a single interrupt into a single larger packet (up to 64KB) so that the network stack has to process fewer headers, resulting in a 10% to 30% reduction in I/O overhead depending on the workload, thereby improving performance. Receive Segment Coalescing (RCS) is able to collect packets that are received during the same interrupt cycle and put them together so that they can be more efficiently delivered to the network stack. This can significantly increase the amount of traffic that can be handled without severely impacting the CPU.

Possible states: enabled, disabled, default
Default state: disabled
Recommended: enabled for better throughput, disabled for pure gaming to ensure lowest latency.

To enable using netsh:

```
netsh int tcp set global rsc=enabled
```

To enable using PowerShell cmdlets:

Enable-NetAdapterRsc -Name * (use to enables RSC for all adapters that support it)
Disable-NetAdapterRsc -Name * (use to disable RSC for all adapters)
Get-NetAdapterRsc -Name * (use to view adapters that support RSC)

Notes:
Needs *Checksum Offload* to be enabled. Only supported by some network adapters.
For gaming, where latency is more important than pure throughput, any type of packet/memory coalescing should be disabled or used very sparingly. Packet/memory coalescing reduces CPU utilization and increases throughput, however, it causes the network adapter to combine packets before interacting with other hardware, which may slightly increase latency. Also see our gaming tweaks article.

Large Send Offload (LSO)

This setting enables Large Send Offload. When enabled, the network adapter hardware is used to complete data segmentation, theoretically faster than operating system software. Theoretically, this feature may improve transmission performance, and reduce CPU load. LSO reduces the interrupt rate and the amount of bus transactions to the NIC. The problem with this setting is buggy implementation on many levels, including Network Adapter Drivers. Intel and Broadcom drivers are known to have this enabled by default, and may have many issues with it. It should only be enabled with the newest Gigabit NICs with recent drivers. Because of the issues mentioned above, we recommend disabling LSO at both the Network Adapter properties, and at the OS level with the setting below.

Default: adapter-dependent
Recommended: disable (both in network adapter properties and in the TCP/IP stack at the OS level)

Disable-NetAdapterLso -Name * (disable LSO for all visible network adapters)
Enable-NetAdapterLso -Name * (enable LSO for both IPv4 and IPv6 on all network adapters. Be careful with this setting, test before using in production)
Get-NetAdapterLso -Name * (get a list of network adapters that support LSO)

Notes: Default state is network adapter dependent. Needs *Checksum Offload* to be enabled to work. LSO is another buffer that may impact latency, it is not recommended for interactive connections and gaming.
See Also: Should I enable TCP Offloading ?

ECN Capability

ECN (Explicit Congestion Notification, RFC 3168) is a mechanism that provides routers with an alternate method of communicating network congestion. It is aimed to decrease retransmissions. In essence, ECN assumes that the cause of any packet loss is router congestion. It allows routers experiencing congestion to mark packets and allow clients to automatically lower their transfer rate to prevent further packet loss. Traditionally, TCP/IP networks signal congestion by dropping packets. When ECN is successfully negotiated, an ECN-aware router may set a bit in the IP header (in the DiffServ field) instead of dropping a packet in order to signal congestion. The receiver echoes the congestion indication to the sender, which must react as though a packet drop were detected. ECN is disabled by default in modern Windows TCP/IP implementations, as it is possible that it may cause problems with some outdated routers that drop packets with the ECN bit set, rather than ignoring the bit.

Possible settings are: enabled, disabled, default (restores the state to the system default).
Default state: disabled

Recommendation: "enabled" only for short-lived, interactive connections and HTTP requests with routers that support it, in the presence of congestion/packet loss, "disabled" otherwise (for pure bulk throughput with large TCP Window, no regular congestion/packet loss, or outdated routers without ECN support).

To change using netsh:

netsh int tcp set global ecncapability=default

To change using PowerShell cmdlets:

Set-NetTCPSetting -SettingName InternetCustom -EcnCapability Disabled

Notes:
Alternative netsh syntax is: *netsh int tcp set global ecn=default*

ECN is only effective in combination with AQM (Active Queue Management) router policy. It has more noticeable effect on performance with interactive connections, online games, and HTTP requests, in the presence of router congestion/packet loss. Its effect on bulk throughput with large TCP Window are less clear. Currently, we do not recommend enabling this setting, as reportedly it has negative impact on throughput with some residential US ISPs. EA multiplayer games that require a profile logon do not support ECN as well (you will not be able to logon). However, it can also reduce latency in some games with ECN-capable routers in the presence of packet loss (dropped packets).

See also: Wikipedia - ECN, RFC 3168

TCP 1323 Timestamps

RFC 1323 ads many useful options, most notably scaling of the TCP Window. "Timestamps" (TSOpt) is a less commonly used 1323 option that is intended to increase transmission reliability by retransmitting segments that are not acknowledged within some retransmission timeout (RTO) interval. The problem with timestamps is that they add 12 bytes to the 20-byte TCP header of each packet, so turning them on causes considerable overhead.

Possible states: enabled,disabled,default
Default state: disabled
Recommended: disabled

To set using netsh:

netsh int tcp set global timestamps=disabled

To set using PowerShell cmdlets:

Set-NetTCPSetting -SettingName InternetCustom -Timestamps Disabled

Initial RTO and Min RTO

Retransmit timeout (RTO) determines how many milliseconds of unacknowledged data it takes before the connection is aborted. The default timeout for Initial RTO of 3 seconds can

usually be lowered for low-latency modern broadband connections, unless you're in a remote location, on a satellite internet connection, or experiencing high latency. In high-latency situations, this can increase retransmissions if the RTO value is reached on a regular basis.

InitialRTO
Default value: 3000 (3 seconds)
Recommended: between 2000 (2 seconds) and 3000 (3 seconds).

To set using netsh:

netsh int tcp set global initialRto=3000

To set using PowerShell cmdlets:

Set-NetTCPSetting -SettingName InternetCustom -InitialRto 3000

MinRTO
Default value: 300 (ms)
Recommended: 300 (ms)

MinRTO can't be changed under Windows 8 (read-only error), it may be editable under server OSes. The command is:

set-NetTCPSetting -SettingName InternetCustom -MinRto 300

Non Sack Rtt Resiliency (Windows 8.1)

Enables/Disables RTT resiliency for non SACK clients. This can help slow clients/connections as it makes TCP/IP less aggressive in retransmitting.

Possible states: enabled,disabled,default
Default state: disabled
Recommended: disabled

To set using netsh:

netsh int tcp set global nonsackrttresiliency=disabled

To set using PowerShell cmdlets:

Set-NetTCPSetting -SettingName InternetCustom -NonSackRttResiliency disabled

Note: This property only supported in Windows 8.1 and Windows Server 2012 R2.

Max SYN Retransmissions (Windows 8.1)

Sets the number of times to attempt to reestablish a connection with SYN packets.

Possible values: 2 to 8
Default value: 2
Recommended: leave at 2

To set using netsh:

netsh int tsp set global maxsynretransmissions=2

To set using PowerShell cmdlets:

Set-NetTCPSetting -SettingName InternetCustom -MaxSynRetransmissions 2

To view the current setting under PowerShell:

Get-NetTCPSetting -SettingName InternetCustom
Or, to view only the current MaxSynRetransmissions setting as a number:
(Get-NetTCPSetting -SettingName InternetCustom).MaxSynRetransmissions

Note: This property only supported in Windows 8.1 and Windows Server 2012 R2. Windows 7 hotfix displays it in netsh as a read-only setting.

InitialCongestionWindow (ICW)

When creating a TCP connection, the sending side performs a "TCP slow start" regardles of the receiver RWIN value. TCP slow start only sends two frames, waits for ACK response, and increases speed exponentially provided there are no dropped packets. This slow start algorithm can also be activated if there is no traffic for 200ms. This is not optimal for fast internet connections with intermittent bursts of data. This bottleneck can be avoided by increasing the "InitialcongestionWindow" from the default 2 (or 4) frames to 10 (See RFC 3390 and RFC 6928).

The following should work for 2012 server, however it does not work in Windows 8.1 (read-only setting error):

Set-NetTCPSetting -SettingName InternetCustom -InitialCongestionWindow 10

The following command works for Server 2008 R2 (with hotfix KB2472264 linked below):

netsh int tcp set supplemental template=custom icw=10

Setting MTU

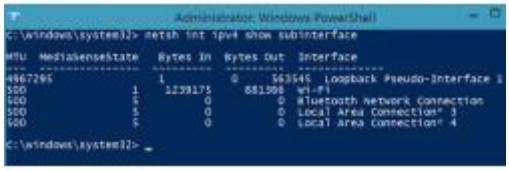
It is sometimes useful to view and set the Maximum Transmission Unit (MTU) value for a specific network adapter manually. To view a list of active network interfaces, their names and their respective MTU values in Windows 8, open PowerShell (or command prompt) as administrator and run the following command:

netsh int ipv4 show subinterface

or, if you are using IP version 6:
netsh int ipv6 show subinterface

Alternative netsh syntax: *netsh int ipv4 show interfaces*
Alternative PowerShell syntax: **Get-NetAdapter** (to view all advanced properties: *Get-NetAdapterAdvancedProperty*)

You will be presented with a list of interfaces, and their respective MTU values as follows:



To change the MTU value of a specific network card, type the following:

netsh int ipv4 set subinterface "network interface name" mtu=#### store=persistent

Where "network interface name" is your specific network adapter name as obtained above (also viewable under Network adapters), and mtu=#### is the desired MTU value. For example, if the name of your network card is "Wi-Fi" and you'd like to set its MTU to 1500, you'd have to type:

netsh int ipv4 set subinterface "Wi-Fi" mtu=1500 store=persistent

Notes:
For IPv6, you can set the MTU using: netsh int ipv6 set subinterface "Wi-Fi" mtu=1500 store=persistent
The maximum MTU value is usually 1500, and up to 1492 for PPPoE connections.
The PowerShell cmdlet using Set-NetAdapterAdvancedProperty SetMTU doesn't seem to work yet. See this nice attempt to set MTU using PowerShell at Serverfault.

Registry Tweaks

This section lists a number of TCP/IP related Windows Registry parameters that work with Windows 8/8.1 and 2012 Server. Some of the settings we've tested are the same/similar parameters as with previous Windows versions that we have confirmed to work with Windows 8/8.1 as well, although they may have different default values. Below is a list of what we've confirmed to work. To edit the Windows Registry, click the Windows button, and type: *regedit* in the search dialog (you may have to right-click *regedit* and choose to run as administrator to have the proper write permissions). It is always a good idea to create a restore point and backup the registry before making changes.

DefaultTTL

TTL does not directly affect speed, and can be safely left alone in many cases. It is a limit to the time and number of hops/routers a packet will travel before being discarded. A number that's too small risks packets being discarded before reaching their destination. A number that's too large (over 128) will cause delay in when lost IP packets are discarded. The default, when the setting is not present in the Windows 8/10/2012 registry the system assumes a value of 128.

HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters
DefaultTTL=64 (DWORD, not present by default denoting 128, valid range is 1-255, recommended: 64)

Host Resolution Priority Tweak

This is intended to increase the priority of DNS/hostname resolution, by increasing the priority of four related processes from their defaults. It is important to note that this increases their priority compared to the hundreds of other running processes, while keeping their order. See our Host Resolution Priority Tweak article for more details. The registry location under Windows 8, 8.1, 10, 2012 Server, and 2012 R2 is:

HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\Tcpip\ServiceProvider\
LocalPriority = 4 (DWORD, default 499, recommended 4)
HostPriority = 5 (DWORD, default 500, recommended 5)
DnsPriority = 6 (DWORD, default 2000, recommended 6)
NetbtPriority = 7 (DWORD, default 2001, recommended 7)

MaxUserPort and TCPTimedWaitDelay (port allocation)

Short lived (ephemeral) TCP/IP ports above 1024 are allocated as needed by the OS. The Windows 8/2012 defaults are usually sufficient under normal network load. However, under

heavy network load it may be necessary to adjust these two registry settings to increase port availability and decrease the time to wait before reclaiming unused ports.

If the default limits are exceeded under heavier loads, the following error is observed "*address in use: connect exception*". To tweak, add/edit the following keys using regedit:

HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters
MaxUserPort=65534 (DWORD, not in the registry by default, uses ports 1024-5000 by default. recommended: 16384 to 65534 decimal as necessary) - maximum number of ports to use. 1024 is automatically subtracted from entered value to allow for reserved ports under 1024.

TcpTimedWaitDelay=30 (DWORD, not present in registry by default, 120 seconds when not present. Recommended: 30 decimal, denoting 30 seconds) - time to wait before reclaiming ports, in seconds. Default time before reclaiming ports, if value is at 0xffffffff or not present in the registry is 120 (or 240) seconds, depending on your OS. Just reducing the delay is often sufficient without changing MaxUserPort, as it allows for reusing ports more efficiently.

See:
MSDN ms737757
Technet MaxUserPort

QoS Reserved Bandwidth

The Windows "QoS Packet Scheduler" under Windows 8/8.1 (as with previous Windows versions) reserves 20% of bandwidth by default for QoS-aware applications that request priority traffic. Note this only has effect in the presence of running QoS applications that request priority traffic, like Windows Update, for example. To prevent QoS applications from getting priority to 20% of available bandwidth, customize the following registry setting:

HKEY_LOCAL_MACHINE\SOFTWARE\Policies\Microsoft\Windows\Psched
NonBestEffortLimit=0 (DWORD, not present in the registry by default. When not present it reserves 20% of bandwidth. Recommended: 0 , possible values between 0 and 100) - indicates the percentage value of reserved bandwidth for QoS applications. Set to 0 to disable.

Notes:
The NonBestEffort tweak applies only when the Windows Qos Packet Scheduler is enabled, and has effect ONLY in the presence of running QoS applications.
QoS reserved bandwidth can also be disabled from the Gourp Policy Editor: Start -> Run: gpedit.msc -> open Local Group Policy Editor\Local Computer Policy\Computer Configuration\Administrative Templates\Network\QoS Packet Scheduler -> in the right pane, edit/enable "Limit reservable bandwidth" and reduce the percentage of "Bandwidth limit" to zero.

QoS Policy

QoS Policy settings under Windows 7/8 can be edited using the Group Policy Editor (gpedit.msc): Computer Configuration -> Windows Settings -> Policy-based QoS

In order to define DiffServ (DSCP) values, according to Microsoft the machine needs to have joined a domain, and interfaces have to see the domain controller. To overcome this limitation, so that you can tag DSCP values even for adapters that do not have access to a domain, use the following hidden registry key:

HKEY_LOCAL_MACHINE\System\CurrentControlSet\Services\Tcpip\QoS (you may have to create the QoS key)
"Do not use NLA"="1" (REG_SZ string value, not DWORD, not present by default, recommended: 1 if you plan to edit DSCP values via gpedit.msc)

Notes:
gpedit.msc is only provided with the Professional/Ultimate and Server Windows variants.
This registry key may also be needed in systems with multiple network adapters present.

Network Memory Allocation (Event ID 2017 error)

When using Windows to serve many/large files over the local network, it is possible to sometimes run into memory allocation errors related to the Windows share, especially with clients that use different operating systems. When this happens, you can usually see the following error in the Event Viewer System log:
Event ID: 2017 "The server was unable to allocate from the system nonpaged pool because the server reached the configured limit for nonpaged pool allocations." It is also possible to get an error indicating that "Not enough server storage is available to process this command". To avoid those errors, you need to change the way Windows allocates memory for network services and file sharing. The below settings optimize the machine as a file server so it would allocate resources accordingly. There are two related registry settings:

HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management
LargeSystemCache=1 (DWORD, default value: 0, recommended value: 1)
A value of zero establishes a cache of ~8 MB, a value of 1 allows the cache to expand up to the amount of physical memory minus 4 MB, as necessary.

HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\LanmanServer\Parameters
Size=3 (DWORD, default value: 1, recommended value: 3)
minimizes used memory
balance used memory
optimal setting for file sharing and network applications

Internet Explorer Web Browser Tweaks

MaxConnectionsPerServer / MaxConnectionsPer1_OServer Internet Explorer browser tweaks adjust the maximum number of concurrent connections your web browser can establish to any single website.

HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Internet Explorer\MAIN\FeatureControl\FEATURE_MAXCONNECTIONSPER1_0SERVER\explorer.exe=10 (DWORD, default: 4, recommended: 8, valid range: 2-128)
FEATURE_MAXCONNECTIONSPERSERVER\explorer.exe=10 (DWORD, default: 2, recommended: 8, valid range: 2-128)

In addition to explorer.exe, you can also add a 32bit DWORD "iexplore.exe" (according to MSDN), and set it to the same corresponding value as "explorer.exe"
FEATURE_MAXCONNECTIONSPER1_0SERVER\iexplore.exe=10 (DWORD, not present by default, recommended: 8, valid range: 2-128)
FEATURE_MAXCONNECTIONSPERSERVER\iexplore.exe=10 (DWORD, not present by default, recommended: 8, valid range: 2-128)

For more information, see our Web Browser Tweaks article

Note: Increasing this setting much over 10 can cause incomplete page loading and other issues with some pages.

Network Throttling Index (Gaming)

By default, Windows 8 continues to implement a network throttling mechanism to restrict the processing of non-multimedia network traffic to 10 packets per millisecond (a bit over 100 Mbits/second). The idea behind such throttling is that processing of network packets can be a resource-intensive task, and it may need to be throttled to give prioritized CPU access to multimedia programs. In some cases, such as Gigabit networks and some online games, for example, it is beneficial to turn off such throttling all together for achieving maximum throughput.

HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Multimedia\SystemProfile
NetworkThrottlingIndex=ffffffff (DWORD, default: 10, recommended: 10 for media sharing, ffffffff for gaming and max throughput, valid range: 1 through 70 decimal or ffffffff to completely disable throttling)

It is only recommended to change this setting in saturated Gigabit LAN environments, where you do not want to give priority to multimedia playback. Reportedly, disabling throttling by using ffffffff can also help reduce ping spikes in some online games. Games that may be affected by this throttling: Source Engine games (TF2, Left 4 Dead, CS:S), HoN, CoD, Overlord series.

SystemResponsiveness (Gaming and Multimedia)

In Windows 8/8.1, just like with Windows 7, multimedia applications use the "Multimedia Class Scheduler" service (MMCSS) to ensure priritized access to CPU resources, without denying CPU resources to lower-priority background applications. However, this also reserves 20% of CPU by default for background processes, your multimedia streaming and some games can only utilize up to 80% of the CPU. This setting, in combination with the above "NetworkThrottlingIndex" can help some games and video streaming. We recommend reducing the reserved CPU for background processes from the default of 20%.

Navigate to:

HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Multimedia\SystemProfile
SystemResponsiveness=10 (DWORD, default is 20 denoting 20% of CPU reserved, recommended: decimal 10, or 0 for pure gaming/multimedia performance)

Notes: The number in this key is rounded by MMCSS to the nearest 10. In some server operating systems (Windows 2008 Server), the SystemResponsiveness may be set to 100, instead of 20 by default. This is by design, giving higher priority to background services over multimedia.

See also: MSDN ms684247

Disable Nagle's algorithm (Gaming)

Nagle's algorithm is designed to allow several small packets to be combined together into a single, larger packet for more efficient transmissions. While this improves throughput efficiency and reduces TCP/IP header overhead, it also briefly delays transmission of small packets. Disabling "nagling" can help reduce latency/ping in some games. Keep in mind that disabling Nagle's algorithm may also have some negative effect on file transfers. Nagle's algorithm is enabled in Windows by default. To implement this tweak and disable Nagle's algorithm, modify the following registry keys.

This setting configures the maximum number of outstanding ACKs:
HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters\Interfaces\{NIC-id}
There will be multiple NIC interfaces listed there, for example: {1660430C-B14A-4AC2-8F83-B653E83E8297}. Find the correct one with your IP address listed. Under this {NIC-id} key, create a new DWORD value:
TcpAckFrequency=1 (DWORD value, not present by default interpreted as 2, 1=disable, 2=default, specifies number of outstanding ACKs before ignoring delayed ACK timer). For gaming performance, recommended is 1 (disable). For pure throughput and data streaming, you can experiment with small values over 2. Wifi performance may see a slight improvement with disabled TcpAckFrequency as well.

In the same location, add a new DWORD value:
TCPNoDelay=1 (DWORD value, not present by default, 0 to enable Nagle's algorithm, 1 to disable)

To configure the ACK interval timeout (only has effect if nagling is enabled), find the following key:
TcpDelAckTicks=0 (DWORD value, not present by default interpreted as 2, 0=disable nagling, 1-6=100-600 ms). Note you can also set this to 1 to reduce the nagle effect from the default of 200ms without disabling it.

For Server Operating Systems that have Microsoft Message Queuing (MSMQ) installed, or if you have the MSMQ registry hive present, also add TCPNoDelay to:
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\MSMQ\Parameters
TCPNoDelay=1 (DWORD, not present by default, 0 to enable Nagle's algorithm, 1 to disable)

See also: Gaming Tweaks article

SG TCP Optimizer (version 4 Beta)

The SG TCP Optimizer is a free program that allows for easy tweaking of all the above TCP/IP settings for broadband with a couple of clicks. Note that only version 4 of the prorgam supports Windows 8 (and newer) and works with PowerShell cmdlets. There program is available in our downloads section.

The SG TCP Optimizer is a standalone application that does not require installation, you can just save it to the desktop, right-click -> choose "Run as administrator", and apply the "optimal" settings, or customize them to your liking. More detailed information about all available options is provided in the TCP Optimizer documentation, answers to frequently asked questions are available in the Optimizer FAQ, and personalized help is available through our forums.

References

Technet - Network Adapter Cmdlets
TechNet set-NetTCPSetting
MSDN MSFT_NetTCPSetting class
Increasing TCP Initial congestion Window in Windows 2008 Server R2
MS Hotfix for netsh errors in Windows Server 2008 R2 - MSKB 2472264

See Also

Windows 8 FAQs
Windows 7, Vista, 2008 Tweaks
Firefox / Internet Explorer Browser Tweaks

Notes
Some templates/settings, as well as changing the default NetTransportFilter are no longer editable in client Windows versions (Windows 8, 8.1), only Server variants (Server 2008, Server 2012/2012 R2).

The tweaks were tested and confirmed to work with Windows 10 Technical Preview build 10041.

Feedback and suggestions are always appreciated.

User Reviews/Comments:

rate:

avg:

by drbaltazar - 2014-12-30 12:47
FOR set-NetTCPSetting -SettingName InternetCustom -CongestionProvider value must be:

Value Meaning

0 Default
2 Compound TCP (CTCP)
3 Datacenter TCP (DCTCP)

http://msdn.microsoft.com/en-us/library/hh872451(v=vs.85).aspx

by drbaltazar - 2015-01-22 14:02
TCP CHIMNEY OFFLOAD ,it need to be in automatic (why?because it set a value of 16mb wich will be best for most situation)

by Nuck Chorris - 2015-02-18 07:28
What about registry tweaks?

by anonymous - 2015-06-03 07:44
powershell timestamp has 2 possible value: enabled or disabled (window 8.1)

by anonymous - 2015-06-06 13:42
You must have MSMQ installed for TCPNoDelay.

by anonymous - 2015-08-25 06:59
disabling nagle works only for TCP or both tcp and UDP?

by Philip - 2015-08-25 07:21
Nagle's algorithm applies to TCP packets only. It interacts badly with TCP Delayed Acks. It was designed to buffer data (small 1-byte keystrokes, for example) until a full packet's worth is collected before sending it out.

by anonymous - 2015-08-25 14:05
So disabling nagle basically improves TCP and most games (especcially fps games) are using UDP. Improving TCP have negative impact on UDP. Is there any tweaks to improve UDP?

by Philip - 2015-08-25 15:47
Improving TCP does not have negative impact on UDP (only in edge scenarios, when you use up all your upstream bandwidth, lack of any QoS, etc.)
Many games use TCP as well, disabling Nagling is well documented and tested for reducing latency in some games.
Check the Gaming Tweaks article.

by anonymous - 2015-08-25 16:46
Really? So explain me than this testing:

https://www.isoc.org/inet97/proceedings/F3/F3_1.HTM

by Philip - 2015-08-25 17:20
If you use up all your bandwidth (regardless of the protocol), you will experience delays and other congestion issues. That does not mean that tweaking one protocol (on your end) has negative impact on another, merely that TCP congestion on any node/router on the network will have negative impact on UDP as well. We can discuss this in more detail on the forums if you'd like.

by anonymous - 2015-09-22 15:13
Thank you for the info, this info fixed one of the issues I had

by drbaltazar - 2016-01-19 10:36
make sure the cable modem you are provided by your cable provider or by yourself is set to the supported speed by the device (if you got multiple device the slowest is the setting you set to! exemple: the cable box for internet supplied by my cable provider is a rca dcm 425 . a look on the web say that these support only 100mb/s duplex . so set it to 100
and everything should be smoother . most cable provider dont look too much into this (wich is a shame)but you should have a smoother experience even if set lower then what you wish the modem could do .why? one device say 1000 mb/s but the other box down say 100mb/s server dont know what is going so somewhere something is gona back pedal furiously!

by drbaltazar - 2016-03-15 11:00
https://msdn.microsoft.com/en-us/library/windows/hardware/ff546360(v=vs.85).aspx HEY PHILIP sorry to make you work .could you verify this ?ms seem to say :TCP chimney offload is depracated and shouldnt be used?

by drbaltazar - 2016-03-15 11:06
PLEASE REMOVE SINCE ms changed

by Philip - 2016-03-15 12:11
You are correct, we've changed the "recommended" optimal setting for TCP Chimney Offload to disabled since TCP Optimizer 4.0.4 because Microsoft now considers it deprecated and it causes issues in some database servers / multi-threaded environments.

Related Articles:

- Network adapter brand affects latency
- TCP Optimizer 4 Documentation - Windows 7, 8, 10, 2012 Server
- TCP Optimizer Revision History
- Wireless Antenna Guide
- Wireless Network Speed Tweaks

Copyright © 1999-2016 Speed Guide, Inc. All rights reserved.
About · Advertising · Terms of Use · Privacy &